EX10 - Ecuacion del cohete con funciones - Defaults y keywords

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex10_rocket_equation_function.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Encapsular la ecuacion del cohete en una funcion e introducir un parametro por defecto g0.

Variables / elementos del problema

Nombre	Significado
Isp	impulso especifico
m0	masa inicial
mf	masa final
g0	gravedad estandar

Ecuacion o logica en papel

```
delta_v = Isp * g0 * ln(m0/mf)
```

Comprobacion manual

Con Isp=250, m0=500000, mf=250000 y g0=9.80665, delta_v es aproximadamente 1699.6 m/s.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- def
- return
- None
- llamadas

Nuevo en este ejercicio:

- parametro por defecto
- argumentos posicionales
- argumentos keyword
- override de default
- casos de prueba

3. Conceptos Python nuevos

1. Default

Sintaxis: `g0 = 9.80665`

En la definicion significa: si el caller no proporciona g0, usa 9.80665.

2. Posicional

Sintaxis: `calculate_rocket_delta_v(Isp, m0, mf)`

Los argumentos se asignan a parametros por orden.

3. Keyword

Sintaxis: `calculate_rocket_delta_v(Isp=500, m0=500000, mf=250000)`

Los argumentos se asignan explicitamente por nombre.

4. Codigo paso a paso

Paso 1. Definir interfaz

```
def calculate_rocket_delta_v(Isp, m0, mf, g0=9.80665):
```

Isp, m0 y mf son obligatorios; g0 puede omitirse.

Paso 2. Validar

```
if Isp < 0:
    return None
elif m0 <= 0:
    return None
...
```

Cada return corta la ejecucion de esa llamada si una condicion invalida se cumple.

Paso 3. Calcular

```
mass_ratio = m0 / mf
delta_v = Isp * g0 * np.log(mass_ratio)
return delta_v
```

Solo se llega aqui si todas las validaciones anteriores fueron False.

Paso 4. Llamada posicional

```
delta_v = calculate_rocket_delta_v(Isp, m0, mf)
```

g0 no se pasa y por eso usa su default.

Paso 5. Llamada keyword

```
calculate_rocket_delta_v(Isp=500, m0=500000, mf=250000)
```

Los nombres hacen visible que valor corresponde a cada parametro.

Paso 6. Probar entradas invalidas

```
calculate_rocket_delta_v(Isp, 250000, mf)
calculate_rocket_delta_v(Isp, m0, 0)
```

La misma funcion sirve para comprobar como responde a distintos casos.

5. Flujo de ejecucion

```
define function with default
 -> call
 -> validate
 -> compute
 -> return value or None
```

6. Resultado / comprobacion esperada

La llamada nominal calcula ~1699.6 m/s. Las llamadas invalidas imprimen sus mensajes y devuelven None.

7. Errores tipicos al empezar

- Poner un parametro obligatorio despues de uno con default.
- Confundir default con keyword.
- No entender que return corta la funcion.
- No revisar None.

8. Si solo recuerdas cinco cosas

- Un default hace opcional un parametro en la llamada.
- Posicional se asigna por orden.
- Keyword se asigna por nombre.
- return termina la llamada actual.
- Una funcion puede reutilizarse para muchos casos.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.